

TIPE — Tournees de collecte

Du nuage de points au meilleur cycle : code & algorithmes

Probleme du voyageur de commerce (TSP)
applique a une collecte de dechets a Meknès

Jeu de donnees : 43 points (1 depot + arrets)
3 algorithmes compares : plus proche voisin, 2-opt, Held-Karp

Pipeline :

```
points.geojson -> step1 -> matrice des temps  
matrice -> step2 -> resolution & comparaison  
meilleur tour -> step3 -> carte  
points -> step4 -> graphe pondere + matrice d'adjacence
```

1. Le probleme et les noeuds

On modelise une tournée de collecte par le PROBLEME DU VOYAGEUR DE COMMERCE (TSP, Travelling Salesman Problem). Un camion part d'un depot, doit passer une seule fois par chaque point de collecte, puis revenir au depot. On cherche le CYCLE de duree totale minimale.

Les noeuds (sommets du graphe)

Chaque point de collecte est un NOEUD. Les coordonnees viennent du fichier points.geojson (exporte de geojson.io), au format [longitude, latitude]. Regle essentielle : le PREMIER point du fichier est le DEPOT (depart et arrivee du cycle), il porte l'indice 0.

Jeu de donnees courant : 43 noeuds (le depot n0 + 42 arrêts). Les coordonnees, dans l'ordre, sont stockees dans coords.csv.

Les aretes (le graphe est complet)

Entre deux noeuds quelconques on peut toujours se deplacer : le graphe est COMPLET. Le poids d'une arete est le TEMPS de trajet estime. Pour 43 noeuds il y a $n(n-1)/2 = 903$ aretes non orientees : on ne peut pas toutes les dessiner lisiblement, mais la MATRICE D'ADJACENCE les contient toutes (voir derniere section, figures).

2. Les etapes du code (pipeline)

Le projet est decoupe en scripts independants, chacun produisant des fichiers lus par le suivant.

step1_build_matrix.py — construire la matrice des temps

Entree : points.geojson. Sortie : travel_time_matrix.csv (matrice $n \times n$ en SECONDES) et coords.csv. Deux methodes au choix (parametre METHOD) :

- METHOD = "hypothesis" (default) : HYPOTHESE FORTE. Le temps entre deux points = distance a vol d'oiseau / vitesse moyenne constante. Distance calculee par la formule de HAVERSINE (sur la sphere). Aucun reseau, aucun acces Internet, instantane.
- METHOD = "osm" : reseau routier reel via OSMnx. Le temps = plus court chemin (Dijkstra) sur le vrai graphe des rues, ou chaque troncon a une vitesse selon son type. Plus fidele, mais lourd (Internet + dependances).

Formules clefs du mode hypothese :

```
v_ms      = AVG_SPEED_KMH / 3.6          # km/h -> m/s
d          = haversine_m(i, j) * DETOUR_FACTOR
T[i][j]    = d / v_ms                     # temps = distance / vitesse
```

DETOUR_FACTOR = 1.0 correspond au vol d'oiseau pur (hypothese la plus forte) ; ~1.3 approche grossierement les detours routiers.

step2_solve.py — resoudre et comparer

Entree : travel_time_matrix.csv. Lance les trois algorithmes, mesure le temps de chaque cycle ET le temps de calcul, puis ecrit results.csv et best_tour.json. Si SYMMETRIZE = True,

la matrice est rendue symetrique ($T[i][j] <- (T[i][j]+T[j][i])/2$) : necessaire pour que le 2-opt soit justifie. Held-Karp n'est lance que si $n \leq \text{HELD_KARP_MAX}$ (15 par default), car son cout explose.

step3_plot_map.py — visualiser le meilleur tour

Entree : best_tour.json + coords.csv (+ road_graph.graphml si dispo). Si le graphe routier existe (mode osm), trace l'itineraire reel le long des rues ; sinon trace un SCHEMA (segments droits dans l'ordre de visite). Sortie : best_tour.png.

step4_graph.py — graphe pondere + matrice d'adjacence

Entree : coords.csv. Construit le graphe (networkx), place chaque noeud a sa vraie position géographique et etiquette les aretes par leur distance. Sorties : graph_view.png (les K plus proches voisins, pour la lisibilite), adjacency_heatmap.png (toute la matrice $n \times n$ en carte de chaleur) et adjacency_matrix_m.csv (distances en metres).

tsp_core.py — la bibliotheque d'algorithmes

Module importe par step2. Contient les trois algorithmes plus des utilitaires (lecture de matrice, symetrisation, longueur d'un tour, force brute de verification). Aucun acces reseau.

```
def tour_length(tour, D):
    n = len(tour)
    return sum(D[tour[i]][tour[(i+1) % n]] for i in range(n))
```

3. Algorithme 1 — Plus proche voisin (glouton)

Idee : partir d'un sommet, et a chaque etape aller vers le point NON ENCORE VISITE le plus proche. Simple et tres rapide, mais GLOUTON : il ne revient jamais sur ses choix et peut se piéger (les derniers sauts sont parfois tres longs).

Pseudocode

```
tour = [depart] ; visite[depart] = vrai ; courant = depart
repete (n-1) fois :
    j* = argmin_{ j non visite } D[courant][j]
    ajouter j* au tour ; visite[j*] = vrai ; courant = j*
renvoyer tour
```

Complexite et variante

- Cout : $O(n^2)$ pour un depart fixe.
- nearest_neighbor_best_start : on relance depuis CHAQUE sommet et on garde le meilleur tour -> $O(n^3)$, souvent nettement meilleur.

Sans garantie d'optimalite : c'est une HEURISTIQUE (solution approchee, pas forcement la meilleure).

4. Algorithme 2 — 2-opt (amelioration locale)

Idee : partir d'un tour existant (ici le meilleur plus-proche-voisin) et l'AMELIORER en defaisant les croisements. On choisit deux aretes du tour, on les retire, et on RENVERSE le segment intermediaire pour les reconnecter autrement. Si le nouveau tour est plus court, on le garde. On repete jusqu'a ne plus trouver d'amelioration (OPTIMUM LOCAL).

Le test de gain

Pour les aretes (a,b) et (c,d) du tour, l'echange remplace a-b et c-d par a-c et b-d. Le gain est :

```
delta = ( D[a][c] + D[b][d] ) - ( D[a][b] + D[c][d] )
si delta < 0 : on inverse le segment tour[i..j] (amelioration)
```

Proprietes

- Cout : $O(n^2)$ par passe ; quelques passes suffisent en pratique.
- SUPPOSE une matrice SYMETRIQUE (inverser un segment ne change pas son cout) — d'ou le parametre SYMMETRIZE de step2.
- Donne un OPTIMUM LOCAL : meilleur que le glouton, sans garantie d'etre l'optimum global.

5. Algorithme 3 — Held-Karp (exact)

Resolution EXACTE par PROGRAMMATION DYNAMIQUE. On note $C[(S, j)]$ le cout du plus court chemin qui part du depot 0, visite exactement l'ensemble de sommets S, et se termine au sommet j. On construit ces valeurs des petits ensembles vers les grands.

Recurrence

```
C[{j}], j] = D[0][j]
C[S, j] = min_{i in S, i != j} ( C[S \ {j}, i] + D[i][j] )
cout optimal = min_j ( C[{1..n-1}, j] + D[j][0] )
```

Les ensembles S sont codes par des ENTIERS (masques de bits) : le bit k vaut 1 si le sommet k est dans S. On reconstruit le tour en remontant les choix (le 'parent' memorise a chaque etape).

Complexite — pourquoi on ne l'utilise que sur petites instances

- Temps : $O(n^2 * 2^n)$. Memoire : $O(n * 2^n)$.
- Croissance EXPONENTIELLE : a $n=43$, $2^{43} \sim 8.8 * 10^{12}$ etats -> impossible. D'ou HELD_KARP_MAX = 15 (au-dela on saute l'algo).
- Usage : sur un sous-quartier de ≤ 15 points, il donne l'OPTIMUM EXACT, qui sert d'ETALON pour mesurer l'ecart (%) des heuristiques.

force brute (verification)

brute_force enumere les $(n-1)!$ tours possibles : utilisable seulement pour $n \leq \sim 10$. Sert a verifier que Held-Karp renvoie bien le meme optimum.

6. Resultats obtenus

Comparaison sur le jeu de donnees courant (43 noeuds, mode hypothese, matrice symetrisee) :

Methode	Temps cycle	Calcul (s)
Plus proche voisin (depart 0)	18 min 12 s	0.0001
Plus proche voisin (meilleur depart)	16 min 26 s	0.0029
2-opt (sur meilleur PPV)	13 min 23 s	0.0007

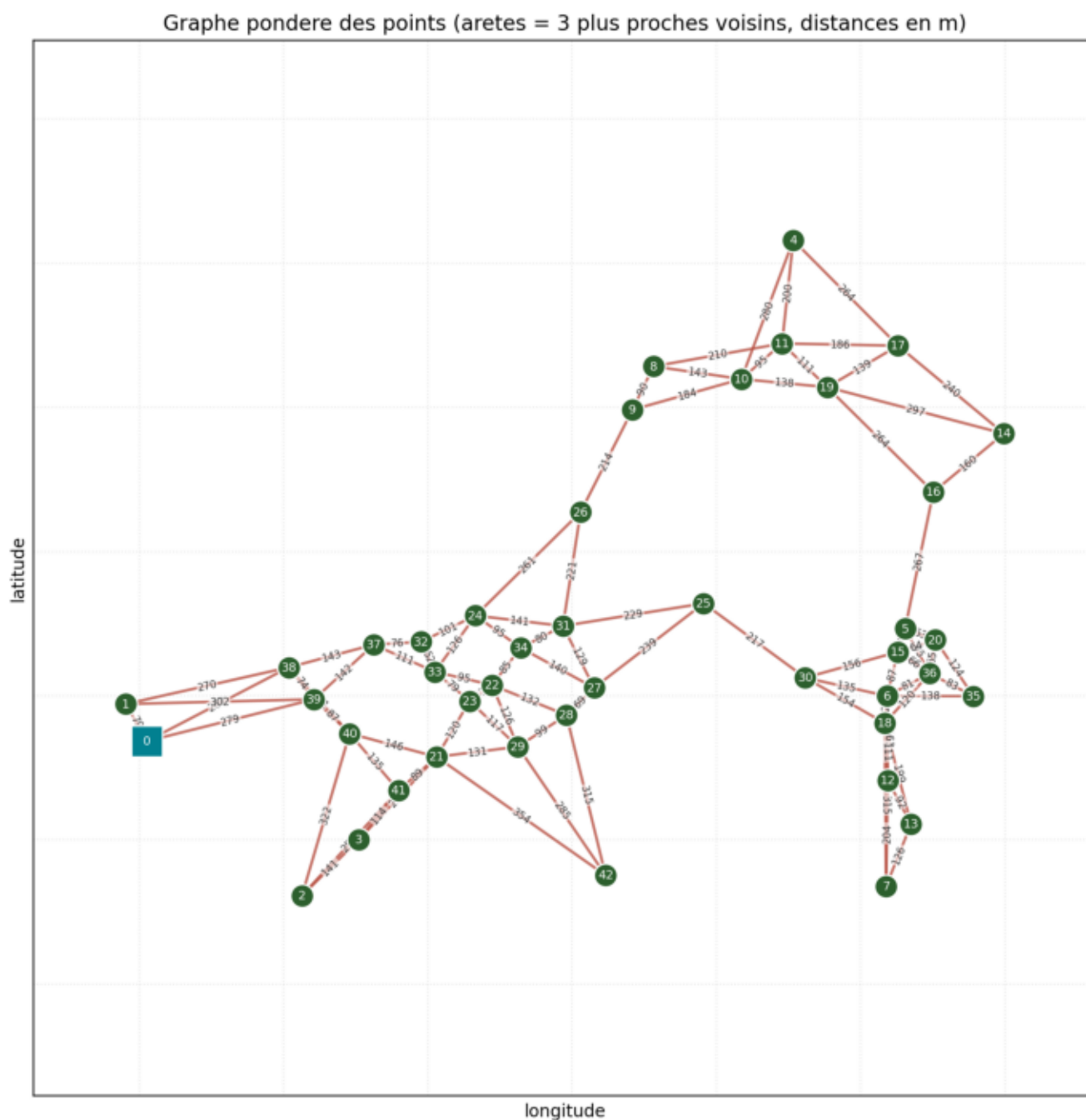
Meilleur cycle : 2-opt (sur meilleur PPV) = 13 min 23 s. Le 2-opt ameliore nettement le glouton : il supprime les croisements que le plus proche voisin laisse en fin de parcours.

Note : Held-Karp est absent ci-dessus car $n > 15$. Pour obtenir l'ecart exact a l'optimum, relancer le pipeline sur un sous-quartier de ≤ 15 points (section 7 du README).

7. Parametres a defendre & limites du modele

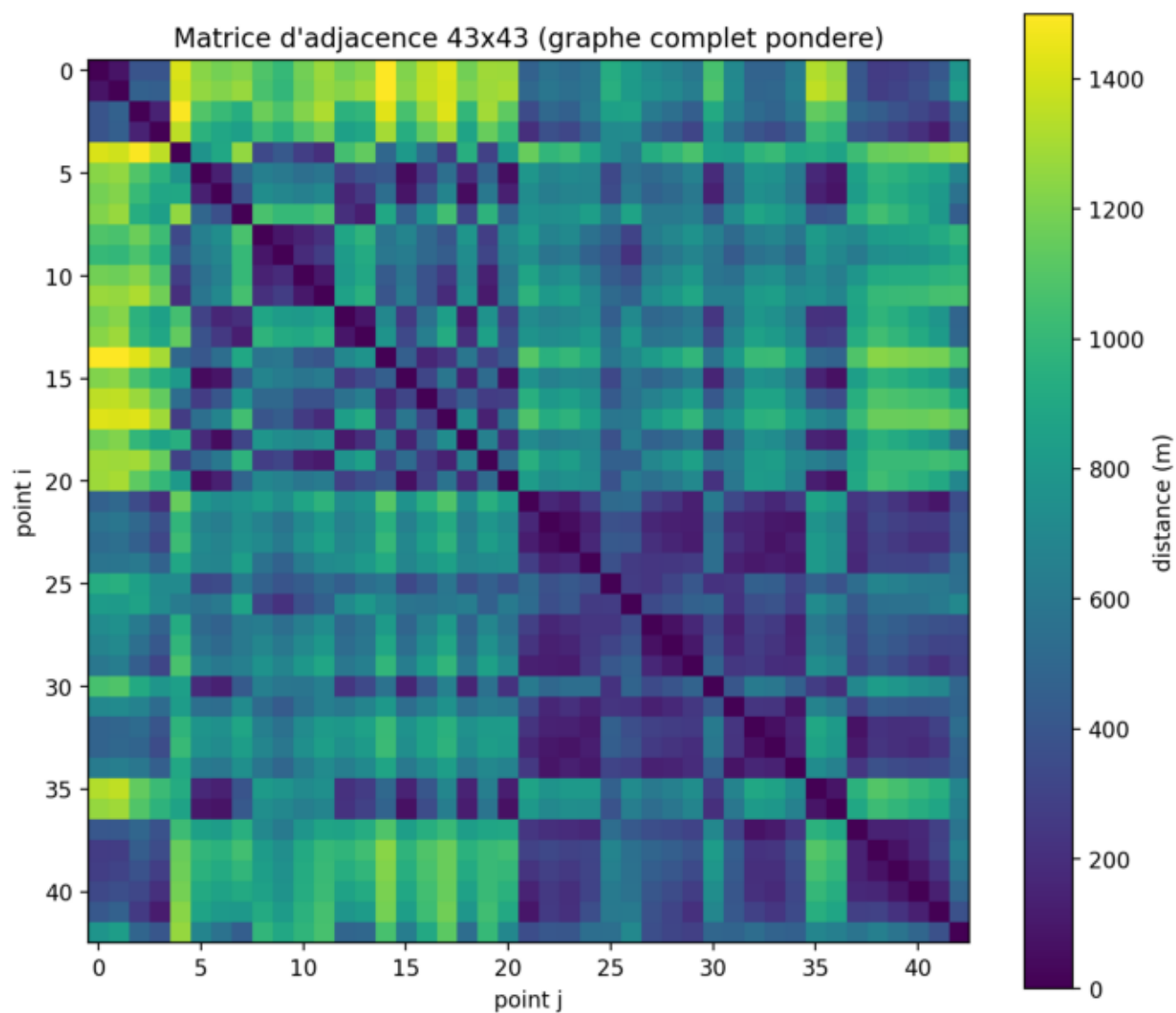
- AVG_SPEED_KMH : vitesse moyenne supposee (~20-30 km/h en ville) — hypothese forte a justifier.
- DETOUR_FACTOR : 1.0 (vol d'oiseau) a ~1.3 (approche les detours).
- SYMMETRIZE : on moyenne les deux sens. Hypothese de modelisation (les sens uniques rendent la vraie matrice asymetrique).
- HELD_KARP_MAX : borne au-dela de laquelle l'exact est saute.
- On resout un TSP ; la vraie collecte est un probleme de tournées sur arcs (CARP/VRP) : simplification assumee.
- Les temps sont des ESTIMATIONS (vitesses libres, sans trafic) : a annoncer comme une premiere approximation / borne.

Figure 1 — Graphe pondere des points



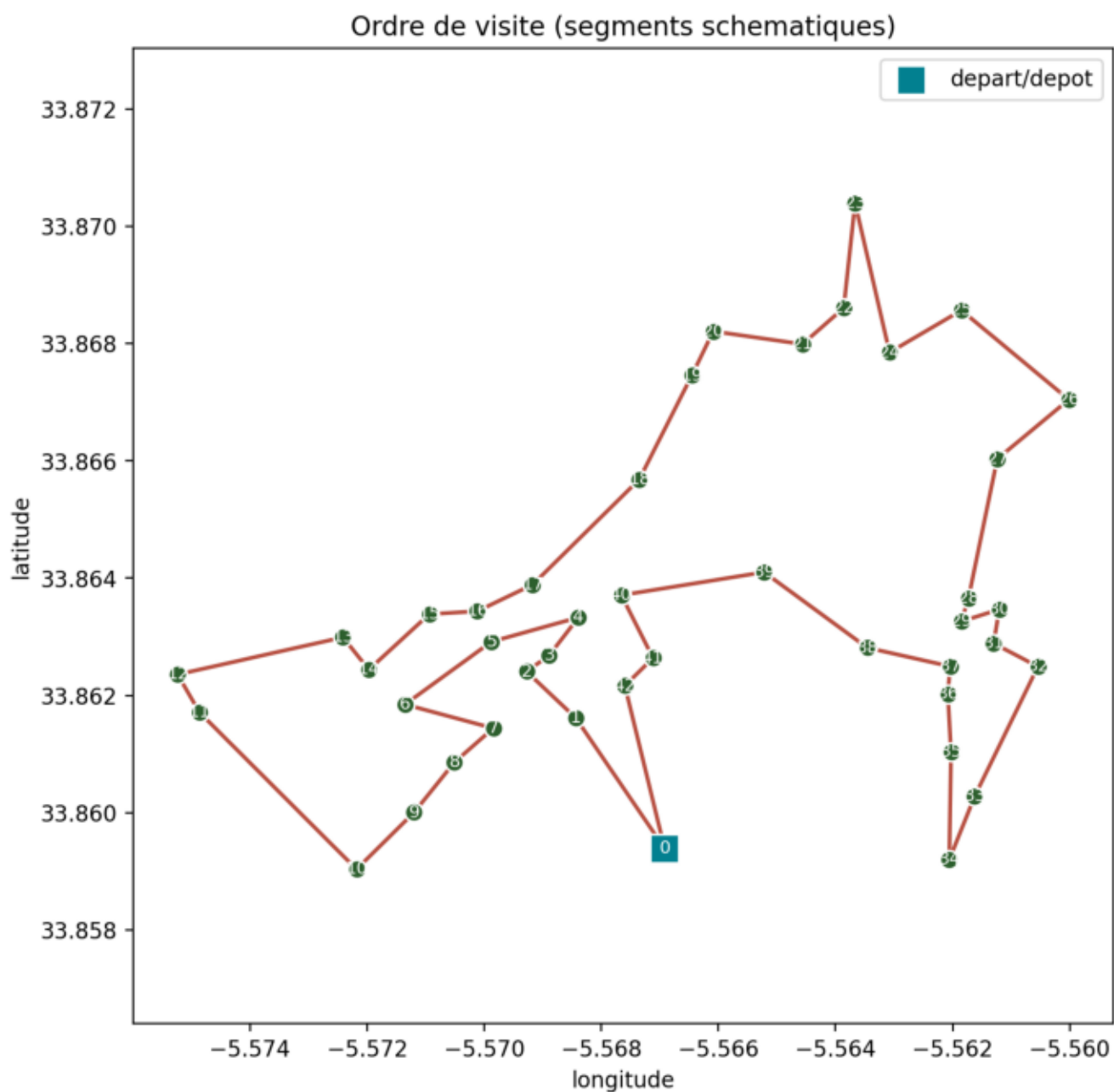
Chaque noeud est place a sa vraie position (lon, lat). Les aretes etiqueetes relient chaque point a ses plus proches voisins ; les etiquettes donnent la distance en metres. Le carre teal est le depot (0).

Figure 2 — Matrice d'adjacence (graphe complet)



La matrice $n \times n$ des distances : sombre = proche, clair = lointain. La diagonale nulle = distance d'un point a lui-meme. Les blocs sombres revelent les groupes de points proches (structure du quartier).

Figure 3 — Meilleur cycle trouve



Le meilleur tour (ici 2-opt) dans l'ordre de visite. Les numeros indiquent l'ordre de passage ; le carre est le depot, depart et arrivee du cycle.